

Image Processing & Recognition: Lecture 2 Review

Welcome to Level 2! In the previous lecture, we built linear models (drawing straight lines). In this lecture, we build **Neural Networks** (drawing complex, curved shapes) to solve harder problems.

Part 1: What are you learning? (The "Human" Summary)

1. The Upgrade: From Linear to "Deep"

A linear model ($\hat{y} = Wx + b$) is limited. It can't solve simple logic problems like XOR (exclusive OR) because you can't draw a straight line to separate the classes.

- **The Solution:** We add "Hidden Layers" between the input and output.
- **The Result:** A **Multilayer Perceptron (MLP)**.
- **Why "Deep"?** The more layers you stack, the "deeper" the network, allowing it to learn hierarchical features (e.g., edges $\rightarrow$ shapes $\rightarrow$ faces).

2. The "Spark": Activation Functions

If you just stack linear layers, the result is still just one big linear equation. To make the network powerful, we apply a **Non-Linear Activation Function** after each layer.

- **ReLU (Rectified Linear Unit):** $f(x) = \max(0, x)$. It turns negative numbers to zero. This is the most popular choice today because it's fast and solves mathematical stability problems.
- **Sigmoid/Tanh:** S-shaped curves used in older networks (or specifically for probabilities).

3. The Battle: Overfitting vs. Generalization

- **Overfitting:** Your student (the model) memorized the textbook (training data) but fails the exam (test data).
- **The Fixes (Regularization):**
 - **Weight Decay (L_2):** Penalize the model for having huge weights (keeps it simple).
 - **Dropout:** Randomly turn off neurons during training. This forces the network to learn robust patterns, not relying on any single "smart" neuron.

4. The Engine: Backpropagation

How does the network learn?

1. **Forward Pass:** Data flows through $\rightarrow$ Prediction is made.
2. **Calculate Loss:** Compare prediction to reality.
3. **Backward Pass (Backprop):** Use the **Chain Rule** to calculate gradients (derivatives) backwards from the output to the input, telling each weight exactly how much to adjust to reduce the error.

Part 2: The Solved Problem (Decoded)

Let's break down **Exercise 2** from your PDF (Page 120).

The Task: Create a regression MLP with:

- **Input (X):** 3 features ($d = 3$). We have 2 examples ($n = 2$).
- **Hidden Layer:** 2 units ($h = 2$). Activation: ReLU.
- **Output Layer:** 1 unit ($q = 1$).
- **Goal:** Calculate the output O and count the total parameters.

The Inputs:

$$X = \begin{bmatrix} 1 & 0 & 1 \\ 1 & -1 & 0 \end{bmatrix} \quad (2 \text{ rows samples, } 3 \text{ cols features})$$

The Weights (Given in Solution):

- **Hidden Weights ($W^{(1)}$):** 3×2 matrix.
- **Hidden Bias ($b^{(1)}$):** 1×2 vector.
- **Output Weights ($W^{(2)}$):** 2×1 matrix.
- **Output Bias ($b^{(2)}$):** 1×1 vector.

$$W^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \end{bmatrix}, \quad b^{(1)} = [1 \quad 0]$$

$$W^{(2)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad b^{(2)} = [1]$$

Step 1: Calculate Hidden Linear Output (Z)

Formula: $Z = XW^{(1)} + b^{(1)}$

- Multiply Input (2×3) by Weights (3×2) $\rightarrow$ Result (2×2).
- Add Bias (1×2) to every row.

$$\begin{aligned} XW^{(1)} &= \begin{bmatrix} 1 & 0 & 1 \\ 1 & -1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \end{bmatrix} = \begin{bmatrix} (1 \cdot 1 + 0 \cdot 0 + 1 \cdot 1) & (1 \cdot 0 + 0 \cdot 1 + 1 \cdot (-1)) \\ (1 \cdot 1 + (-1) \cdot 0 + 0 \cdot 1) & (1 \cdot 0 + (-1) \cdot 1 + 0 \cdot (-1)) \end{bmatrix} \\ &= \begin{bmatrix} 2 & -1 \\ 1 & -1 \end{bmatrix} \end{aligned}$$

Now add bias $[1, 0]$ to each row:

$$Z = \begin{bmatrix} 2+1 & -1+0 \\ 1+1 & -1+0 \end{bmatrix} = \begin{bmatrix} 3 & -1 \\ 2 & -1 \end{bmatrix}$$

Step 2: Apply Activation (H)

Formula: $H = \text{ReLU}(Z) = \max(0, Z)$

- Replace negatives with 0. Keep positives.

$$H = \begin{bmatrix} 3 & 0 \\ 2 & 0 \end{bmatrix}$$

Step 3: Calculate Final Output (O)

Formula: $O = HW^{(2)} + b^{(2)}$

- Multiply Hidden (2×2) by Output Weights (2×1) $\rightarrow$ Result (2×1).
- Add Bias (1×1).

$$HW^{(2)} = \begin{bmatrix} 3 & 0 \\ 2 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 3 \cdot 1 + 0 \cdot 0 \\ 2 \cdot 1 + 0 \cdot 0 \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$$

Add bias $[1]$:

$$O = \begin{bmatrix} 3+1 \\ 2+1 \end{bmatrix} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

Step 4: Count Parameters

- **Layer 1:** (3 inputs $\times$ 2 hidden) + 2 biases = $6 + 2 = 8$.
- **Layer 2:** (2 hidden $\times$ 1 output) + 1 bias = $2 + 1 = 3$.
- **Total:** $8 + 3 = 11$.

Part 3: Your Turn (Interactive Exercises)

Practice Problem 1: The Small Net

Setup:

- **Input X :** 1 sample, 2 features $\rightarrow [2, -1]$.
- **Hidden Layer:** 2 units.
 - Weights $W^{(1)} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$
 - Bias $b^{(1)} = [0, 1]$

- **Output Layer:** 1 unit.
 - Weights $W^{(2)} = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$
 - Bias $b^{(2)} = [3]$
- **Activation:** ReLU.

Task: Find the output O .

Solving Steps (Try it yourself first!):

1. **Hidden Linear (Z):**

$$[2, -1] \times \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = [(2 \cdot 1 + -1 \cdot 1), (2 \cdot 1 + -1 \cdot -1)] = [1, 3]$$

Add bias $[0, 1] \rightarrow [1 + 0, 3 + 1] = [1, 4]$.

2. **Activation (H):** ReLU applied to $[1, 4]$ is just $[1, 4]$ (no negatives to remove).

3. **Output (O):**

$$[1, 4] \times \begin{bmatrix} 2 \\ -1 \end{bmatrix} = (1 \cdot 2) + (4 \cdot -1) = 2 - 4 = -2$$

Add bias $[3] \rightarrow -2 + 3 = 1$.

Final Answer: Output is 1.

Practice Problem 2: Parameter Counting

Setup: You have an MLP designed to process small images.

- **Input:** Flattened 4×4 grayscale image.
- **Hidden Layer 1:** 10 units.
- **Hidden Layer 2:** 5 units.
- **Output Layer:** 3 classes (Cat, Dog, Mouse).

Task: Calculate the total number of trainable parameters (weights + biases).

Solving Explanation:

1. **Input Size:** $4 \times 4 = 16$ features.
2. **Layer 1 Params:**
 - Weights: $16 \text{ inputs} \times 10 \text{ units} = 160$
 - Biases: 10
 - Subtotal: 170
3. **Layer 2 Params:**

- Weights: 10 inputs (from prev layer) $\times$ 5 units = 50
- Biases: 5
- Subtotal: 55

4. **Layer 3 (Output) Params:**

- Weights: 5 inputs $\times$ 3 units = 15
- Biases: 3
- Subtotal: 18

Total: $170 + 55 + 18 = 243$ parameters.